

Configuring a Cluster with group_vars

Kubernetes Production · Lesson 4

Built from the official documentation

`kubernetes-sigs/kubespray` · `docs/ansible/vars.md`

Learning Objectives

By the end of this lesson you will be able to:

- Locate cluster configuration in the **group_vars** directory layout
- Set the **key variables** that define a cluster (version, CNI, runtime, subnets)
- Reason about **Ansible variable precedence** — which value wins
- Configure the **DNS stack** (CoreDNS + nodelocaldns) and upstream DNS
- Configure **NTP**, an **HTTP(S) proxy**, and the **download / pre-pull** mechanism

Topology lives in the **inventory**; everything else lives in **group_vars**.

4.1 The group_vars Layout

Where Configuration Lives

Under `inventory/mycluster/group_vars/`, config is split by Ansible group:

Path	Applies to
<code>all/all.yml</code>	All hosts — proxy, NTP, downloads, LB, cloud
<code>all/*.yml</code>	All hosts — OS, etcd, OpenStack, offline (split files)
<code>k8s_cluster/k8s-cluster.yml</code>	<code>k8s_cluster</code> — version, CNI, runtime, subnets, DNS
<code>k8s_cluster/addons.yml</code>	<code>k8s_cluster</code> — dashboard, ingress, metrics, CSI

`k8s_cluster` = `kube_control_plane` + `kube_node` , so these files reach **every** cluster node.

The Two Files That Matter Most

- `group_vars/all/all.yml` — host-level, cross-cutting concerns that also apply to non-Kubernetes hosts (e.g. the load balancer): `proxy`, `NTP`, `downloads`, `cloud_provider`, `loadbalancer_apiserver`.
- `group_vars/k8s_cluster/k8s-cluster.yml` — the Kubernetes cluster itself: `kube_version`, `kube_network_plugin`, `container_manager`, the service/pod subnets, DNS and kubelet settings.

Kubespray ships **production defaults** — you override only what you need: often just version, CNI, a proxy, and DNS.

4.2 Key Cluster Variables

The Variables That Define a Cluster

Variable	Meaning	Default
kube_version	Kubernetes release	(per release)
kube_network_plugin	CNI: calico / cilium / flannel ...	calico
container_manager	CRI runtime	containerd
kube_service_addresses	Service ClusterIP CIDR	10.233.0.0/18
kube_pods_subnet	Pod IP CIDR	10.233.64.0/18
cluster_name / dns_domain	Cluster DNS zone	cluster.local
kubelet_max_pods	Max pods per node	(110)

k8s-cluster.yml — Core Settings

```
# inventory/mycluster/group_vars/k8s_cluster/k8s-cluster.yml
kube_version: v1.31.4
kube_network_plugin: calico      # cilium | flannel | kube-ovn ...
container_manager: containerd   # cri-o | docker
kube_proxy_mode: ipvs           # iptables | nftables

# Service ClusterIP and Pod CIDRs — MUST NOT overlap
kube_service_addresses: 10.233.0.0/18
kube_pods_subnet: 10.233.64.0/18
kube_network_node_prefix: 24     # per-node pod subnet size

cluster_name: cluster.local
kubelet_max_pods: 110
```


Subnet & Networking Rules

- `kube_service_addresses` and `kube_pods_subnet` **must not overlap** each other, and must not collide with your **node network**.
- `kube_network_node_prefix` (default `24`) sets the per-node pod block — `prefix × node-count` must fit inside `kube_pods_subnet`.
- Host addressing is **per host** in the inventory, not in `group_vars`:
`ip` = service-binding (public) IP, `access_ip` = inter-host (private) IP.
- Some aggregates (e.g. `kube_service_subnets`) are **computed** — set the base variables, not the aggregate.

Plan CIDRs once: changing them after deploy means rebuilding the cluster.

4.3 Variable Precedence

Which Value Wins?

Kubespray uses standard Ansible precedence — **lowest to highest**:

1. Kubespray `role/defaults` — built-in defaults
2. `group_vars/all/*`
3. `group_vars/k8s_cluster/*` — more specific group beats `all`
4. `host_vars` — a single host beats any group var
5. `-e extra-vars` on the CLI — **always wins**

```
# Override for one run without editing any file
ansible-playbook -i inventory/mycluster/inventory.ini \
  -e kube_version=v1.31.4 cluster.yml
```

More specific scope beats more general scope — set a cluster default in `k8s_cluster`, override per node in `host_vars`.

4.4 The DNS Stack

CoreDNS + nodelocaldns

- **CoreDNS** is the default cluster DNS — authoritative for `dns_domain` (`cluster.local`) and its subdomains.
- **nodelocaldns** is a per-node **caching agent**; pods query it locally, avoiding iptables DNAT to the central service IP.
- Enabled by **default since release 2.10** (`enable_nodelocaldns: true`).

Resolution path: **pod** → **nodelocaldns** (local cache) → **CoreDNS** for cluster names → **upstream DNS** for everything else.

dns_mode & Upstream DNS

```
# group_vars/k8s_cluster/k8s-cluster.yml
dns_mode: coredns          # coredns_dual | manual | none
enable_node_local_dns: true # per-node DNS cache (default since 2.10)
resolvconf_mode: host_resolvconf # default

# Resolvers CoreDNS forwards non-cluster queries to
upstream_dns_servers:
  - 1.1.1.1
  - 9.9.9.9
# Extra host nameservers (only with host_resolvconf), after upstream
nameservers:
  - 192.168.0.1
```

- `dns_mode`: `coredns` (default), `coredns_dual`, `manual`, `none`.
- Limits: max **6** search domains; nameservers capped at **3** (~2 custom).

4.5 NTP

Time Synchronization

"Time synchronization is important to Kubernetes and Etcd" — clock skew breaks

TLS certificate validation and **etcd** consensus.

```
# group_vars/all/all.yml
ntp_enabled: true           # install ntpd / chrony, enable at boot
ntp_manage_config: true    # let Kubespray write the config
ntp_servers:
  - "0.pool.ntp.org iburst"
  - "1.pool.ntp.org iburst"
ntp_timezone: Etc/UTC      # unset = leave unchanged
ntp_tinker_panic: true     # avoid large-drift panics in VMs
```

`ntp_manage_config: true` is required to apply `ntp_servers` — essential for **air-gapped** sites pointing at internal NTP.

4.6 HTTP(S) Proxy

Outbound Proxy & Auto no_proxy

```
# group_vars/all/all.yml
http_proxy: "http://proxy.corp.tld:3128"
https_proxy: "http://proxy.corp.tld:3128"
# no_proxy is AUTO-generated (all nodes + loadbalancer)
additional_no_proxy: ".corp.tld,10.0.0.0/8" # append, keep auto
no_proxy_exclude_workers: true # control-plane only
```

- Set only `http_proxy` / `https_proxy`; Kubespray builds `no_proxy` to exclude **all nodes and the loadbalancer**.
- Use `additional_no_proxy` to **append** — overriding `no_proxy` disables auto-generation.
- `no_proxy_exclude_workers: true` keeps only control-plane nodes, so scaling workers won't restart the runtime / bounce pods.

4.7 Downloads / Pre-Pull

Controlling Where Artifacts Come From

```
# group_vars/all/all.yml (or download.yml)
download_run_once: true      # download once, then push to all nodes
download_localhost: false   # true = Ansible node is the delegate
download_force_cache: false # provision from cache, no re-download
download_keep_remote_cache: false
download_cache_dir: /tmp/kubespray_cache
```

- Default: **each node downloads independently** (`download_run_once: false`).
- `download_run_once: true` → delegate is the **first kube_control_plane**; cache (~800 MB) lands in `download_cache_dir`.
- `download_localhost: true` → use the **Ansible control node** as delegate (nodes can't reach external registries); needs a local container runtime.

Key Takeaways

- **Topology = inventory; configuration = `group_vars`**. Edit YAML, run `cluster.yml`; re-edit + re-run for Day-2.
- Two files matter most: `all/all.yml` (proxy, NTP, downloads, cloud) and `k8s_cluster/k8s-cluster.yml` (version, CNI, runtime, subnets, DNS).
- A few variables define the cluster: `kube_version`, `kube_network_plugin`, `container_manager`, `kube_service_addresses`, `kube_pods_subnet`.
- **Precedence:** defaults < `all` < `k8s_cluster` < host_vars < `-e` extra-vars.
- Cross-cutting concerns are first-class: **DNS** (CoreDNS + nodelocaldns), **NTP**, **proxy** (auto `no_proxy`), **downloads** (`download_run_once`).

End of Lesson 4

Next: Lesson 5 — Container Runtimes & CNI

`container_manager: containerd · crio · docker`